AWS Load Balancer Project

Application Load Balancer with Two EC2 Web Servers

1. Objective

Distribute incoming web traffic across two EC2 instances using an Application Load Balancer (ALB), so the website stays available even if one server fails. The project demonstrates load balancing, health checks and basic high availability.

2. Architecture

Users send requests to the ALB DNS name. The ALB listens on port 80 and forwards traffic to a target group that contains two EC2 instances running in different Availability Zones. Health checks remove an unhealthy instance from rotation automatically.

Component	Purpose
VPC + 2 public subnets (2 AZs)	Network for the ALB and instances
Security group: ALB-SG	Allow HTTP (80) from anywhere
Security group: EC2-SG	Allow HTTP (80) only from ALB-SG
EC2 instances (Web-1, Web-2)	Apache web servers showing their hostname
Target group	Registers instances, runs health checks
Application Load Balancer	Receives traffic and routes to healthy targets

3. Prerequisites

• An AWS account (Free Tier is enough for testing; the ALB itself is billed hourly).

• Default VPC with at least two subnets in different Availability Zones.

• Basic familiarity with the EC2 console.

4. Step-by-Step Implementation

Step 1: Create security groups

• **ALB-SG**: inbound HTTP, port 80, source 0.0.0.0/0.

• **EC2-SG**: inbound HTTP, port 80, source = ALB-SG. Optionally allow SSH (22) from your IP only.

Step 2: Launch two EC2 instances

• AMI: Amazon Linux 2023, type t2.micro or t3.micro.

• Launch Web-1 in subnet A and Web-2 in subnet B (different AZs).

• Attach EC2-SG. Paste the following in *Advanced details* > *User data*:

```
#!/bin/bash
dnf install -y httpd
systemctl enable --now httpd
```

```
echo "<h1>Hello from $(hostname -f)</h1>" > /var/www/html/index.html
```

Step 3: Create the target group

• EC2 > Target Groups > Create. Type: Instances. Protocol: HTTP, port 80.

• Health check path: */*. Healthy threshold 2, interval 15 seconds.

• Register Web-1 and Web-2 as targets.

Step 4: Create the Application Load Balancer

• EC2 > Load Balancers > Create > Application Load Balancer.

• Scheme: Internet-facing. Select the same VPC and both subnets.

• Security group: ALB-SG.

• Listener: HTTP : 80, default action forward to your target group.

Step 5: Test

• Wait until the targets show **healthy**.

• Copy the ALB DNS name and open it in a browser.

• Refresh several times: the hostname changes between Web-1 and Web-2.

```
curl http://<ALB-DNS-NAME>
```

5. Failover Test

• Stop Web-1 from the EC2 console.

• After a short time the target group marks it unhealthy.

• Refresh the ALB URL: only Web-2 responds, with no downtime for users.

• Start Web-1 again and watch it return to healthy.

6. AWS CLI Equivalent (Optional)

```
aws elbv2 create-target-group --name web-tg --protocol HTTP --port 80 \
  --vpc-id <vpc-id> --health-check-path /

aws elbv2 register-targets --target-group-arn <tg-arn> \
  --targets Id=<web1-id> Id=<web2-id>

aws elbv2 create-load-balancer --name web-alb --subnets <subnet-a> <subnet-b> \
  --security-groups <alb-sg-id>

aws elbv2 create-listener --load-balancer-arn <alb-arn> --protocol HTTP --port 80 \
  --default-actions Type=forward,TargetGroupArn=<tg-arn>
```

7. ALB vs NLB (Quick Comparison)

Feature	ALB	NLB
OSI layer	7 (HTTP/HTTPS)	4 (TCP/UDP/TLS)
Routing	Path, host, header based	Port/protocol based

Best for	Web apps, microservices	Extreme performance, static IP

8. Cleanup (Avoid Charges)

• Delete the load balancer first, then the target group.

• Terminate both EC2 instances.

• Delete the custom security groups.

9. Conclusion

This project shows how an ALB spreads traffic across multiple servers and keeps the application available when an instance fails. Possible extensions: HTTPS with ACM certificates, Auto Scaling Group instead of fixed instances, and path-based routing rules.